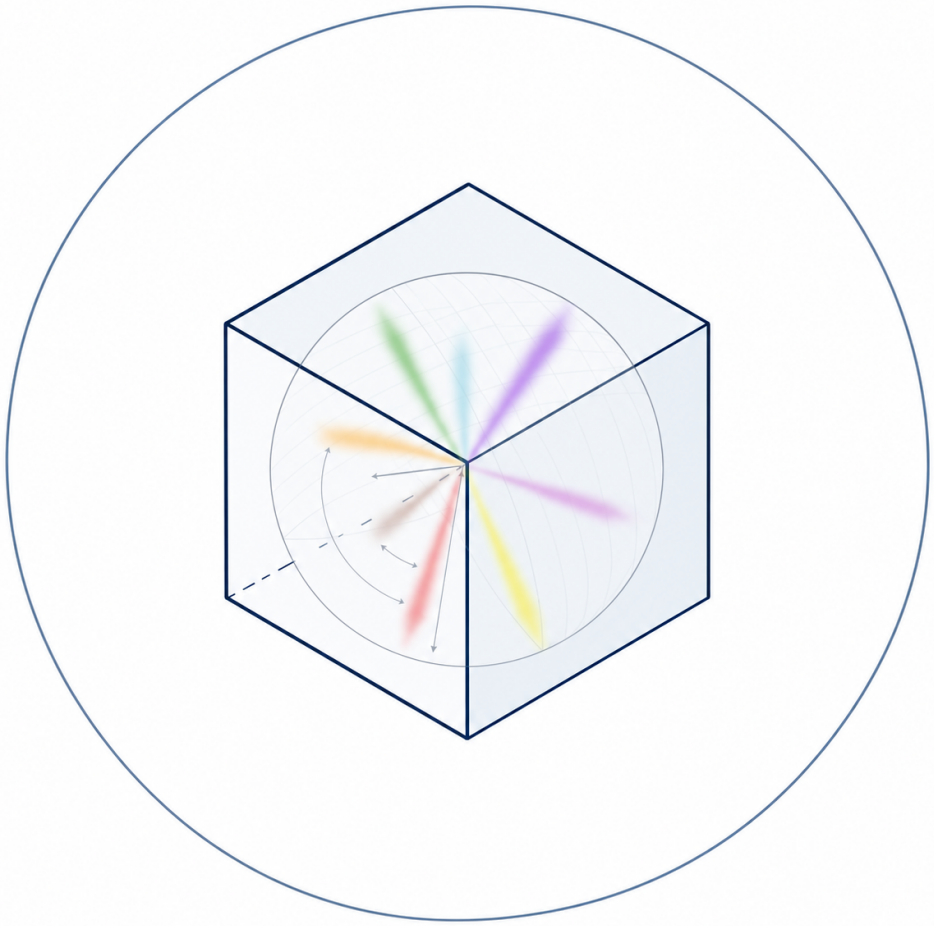
RESEARCH EDITION v1.0

Q-Pathformer

Three-State Architecture and Experimental Specification

Token clusters, vector weights and learning flows



Anthony Siljeg

Alnotrea Labs · Q-Pathformer Initiative

October 2026

Q-Pathformer Three-State Architecture and Experimental Specification

Token clusters, vector weights and learning flows

Anthony Siljeg — Founder, Alnotrea Labs
Company affiliation: Alnotrea Pty Ltd · Q-Pathformer Initiative
Research edition v1.0 · 30 October 2026 · Draft for review

Abstract

Q-Pathformer is a proposed multi-state architecture for organising data preparation, shared representation processing and cluster-level adaptation. State 3 assigns versioned training priorities and schedules examples; State 2 computes representations and selects content routes; State 1 applies adapters and task heads and maintains accepted replay and checkpoint versions. This update defines the priority sampler, inference boundary, training phases and candidate-update protocol. It distinguishes processing classes from prediction labels and separates fixed exposure, fixed updates, fixed resources and time-to-quality comparisons. A complementary two-qubit study maps eight synthetic class IDs to four re-uploading layer counts in a shared parameter bank. Independent matrix checks, analytical invariants and information-matched controls test that study separately. Language modelling and multimodal retrieval remain companion extensions. The proposed mechanisms require implementation and experiments; historical performance figures are not new results.

Edition and evidence. This edition retains the architectural scope of the 18 October 2026 proposal and connects it to subsequent Q-Pathformer studies. Those studies evaluate selected text-classification, circuit-simulation, routing, readout and training-allocation mechanisms under their own conditions. The integrated three-state system and its broader adaptation, language and multimodal extensions remain research objectives. [S4, S5; T01–T10] This document is the architecture companion to the separate Routed States findings paper, which synthesises the completed v0.6, v0.9 and v0.9.5 experiments. [S7]

Provenance. QPATH-ARCH-001 records first publication on LinkedIn on 5 December 2024. Its retained PDF edition was created in March 2026. The companion cover is an editorial adaptation of the original introductory illustration. The original document and its notices are preserved unchanged. [S1]

Reading guide: Architecture · Study evidence · References · Protocol appendices

1 Research problem and contribution

Learning systems make several decisions about where to spend computation. A training procedure chooses which examples to revisit, a routed model selects components or combines their predictions, and an execution policy chooses which experiments receive further budget. Each decision can change predictive quality and cost. Understanding their effects requires controls that separate the mechanism under study from extra model capacity, different information, longer training or a more favourable evaluation.

Conditional-computation research has shown how routing can increase model capacity while limiting the components active for an example. It also exposes practical costs in communication, dispatch and training stability. Benchmarking research makes a related distinction: greater training throughput does not necessarily produce a better time to a quality target. These considerations motivate explicit measurement of both quality and the complete work needed to obtain it. [R2, R10]

Quantum machine learning adds another question: which part of a result depends on the circuit representation, and which part can be explained by data processing, routing or a classical function with comparable information? Broad benchmarking of small quantum classifiers has demonstrated the importance of strong classical controls. Fourier descriptions of parameterised quantum models also provide a useful basis for analysing the functions that particular circuit encodings can represent. These findings motivate careful comparisons; they do not settle the performance of every quantum model. [R15, R14]

Q-Pathformer organises these questions through three responsibilities: preparing examples and training priorities, computing shared representations and routes, and applying specialist components while managing accepted updates. The accompanying studies examine selected mechanisms within this broader proposal. Their results include useful routing and readout effects, prediction-preserving classical implementations of selected trained circuits, and cases where the proposed mechanism does not establish an improvement over a stronger control.

The contribution of this edition is to connect the architecture to that evidence. It makes the tested mechanisms, comparison conditions and remaining hypotheses explicit, so subsequent experiments can address a specific uncertainty. Larger implementations and physical quantum evaluation should follow a defined question and measurable criteria.

This work contributes a three-state research specification, an evidence map connecting that specification to the completed component studies, and a set of explicit comparison and measurement principles for the next experiments. The supporting studies show where geometric routing, richer readouts, training priority or learned routing help under their tested conditions, where strong classical alternatives remain preferable, and where selected circuit predictors admit faster equivalent classical evaluation. These findings narrow the next research questions while preserving the distinction between implementation correctness, predictive benefit and resource advantage. [T01–T10]

This programme does not claim to have invented adapters, mixture-of-experts routing, replay, curricula, Fourier representations or time-to-quality measurement. Those methods provide prior art and controls. [R1, R2, R10–R14]

2 Three-state architecture and terminology

Prediction follows State 3 to State 2 to State 1 once. Training schedules, candidate updates and feedback operate only in the declared training lifecycle. Prediction does not trigger adaptation or checkpoint replacement.

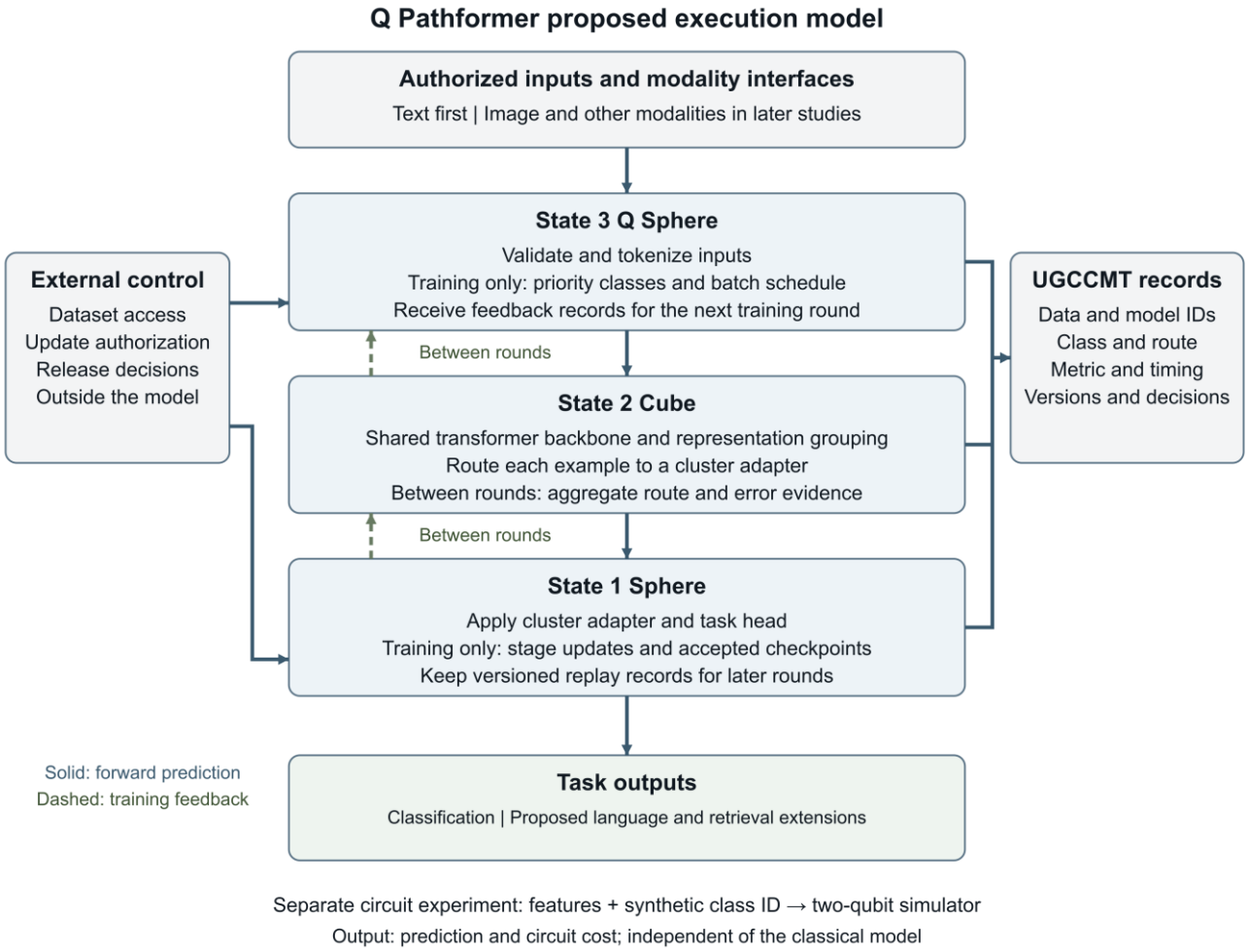


Figure 1 Proposed state responsibilities. Solid arrows show the forward prediction path; dashed arrows show feedback between training rounds. Language and retrieval outputs are proposed extensions. Adapted from the preserved architecture diagram. [S4, S5]

Q-Sphere owns validation, tokenisation and the training priority policy. Cube owns the shared backbone and route selection. Sphere owns adapters, heads, replay and accepted model versions. External decisions govern dataset access and release; experiment-state acceptance below is a research protocol, not a new governance platform. [S1, p. 1]

Table 1 Three architectural responsibilities

State	Name	Responsibility
3	Q-Sphere	Prepare eligible inputs and schedule training examples using declared priorities.
2	Cube	Compute shared representations and select content routes.
1	Sphere	Apply specialist adapters and task heads; manage candidate and accepted updates.

The order is 3 → 2 → 1. These are architectural responsibilities, not physical quantum states. Prediction uses the declared forward path; feedback belongs to a registered training or adaptation lifecycle. [S1, S4, S5]

The nested sphere–cube–sphere image is the historical conceptual representation; the forward-path diagram specifies the proposed division of software responsibilities and does not prescribe a physical geometry or quantum state.

Three decisions about work must remain distinct: training-example sampling chooses records for an update; model routing selects experts or weights their outputs; training-job allocation assigns further budget to competing recipes. The semantic evidence rubric, loss-based priority in v0.8 and suitability allocation in v0.9.5 are different interventions. [T08, §§1–3; T10, §§1–3]

Q-Path Multiplier - Elastic Task Execution Layer (ETEL) v0.1 coordinates bounded tasks and workers, execution records, recovery and resource accounting outside the predictor. It is not a fourth state or another name for the State 3 sampler. Incorporated source 0.2.0 is a historical implementation identifier. Its execution role and an allocation policy require separate evaluations. [T07; T10, §§2–3]

Hard routing in v0.9 evaluates one expert. Dense fixed-soft and learned-soft variants evaluate all five and combine their probabilities. Learning soft weights does not reduce that expert count. Fixed-soft weights depend on the input through a fixed geometric rule. [T09, §2]

3 Questions and evidence from completed studies

The following map connects the proposal to completed component studies. A report remains the source of its precise protocol, comparator, uncertainty and limitations. Report identifiers T01 through T10 are listed in the bibliography.

Table 2 Proposal-to-evidence map

Question	Evidence	Interpretation and boundary
Allocation and text routing	T01–T02, opening findings and primary comparisons	Strong fixed-depth and classical controls remain counterexamples. The primary text differences do not establish a resolved routing improvement. AG News and DBpedia14 are separate tasks.
Useful input geometry	T03–T04; T07, opening findings and routing contrasts	Geometric routing helps specified synthetic-task comparisons. It does not validate the language architecture or a universal routing rule.
Readout and representation	T05–T07, opening findings and classical conversion analyses	Readout and representation change quality. Equivalent classical evaluation can reduce latency for the same fitted predictor; it is not a new efficacy arm.
Training-example priority	T08, p. 1 and primary comparison results	Four of eight direct registered BCE comparisons favour the tested policy; four are inconclusive. None establishes more than 0.01 BCE improvement across its full interval. This is loss-based sampling, not the semantic rubric.
Learning the route	T09, p. 1 and primary contrasts	Gains depend on task and comparator. Spiral and rare-disk gains over fixed routing coexist with strong global-classical controls. The annulus comparison with G121 does not isolate the need for gate learning.

Question	Evidence	Interpretation and boundary
Suitability allocation	T10, p. 1 and registered native-BCE contrasts	None of the eight registered comparisons establishes the stipulated 0.01 native-BCE improvement. Successive halving uses less mean native CPU on all four tasks. Execution and recovery are separate questions.
Integrated architecture	S4–S5; future registered protocols	Component results do not establish adaptation, language-model or multimodal benefit for the complete architecture.

The studies form a sequence of research decisions with different tasks, controls and budgets. They are not paired evidence of continuous version-to-version improvement. The common clipped-BCE comparisons in v0.7–v0.9 and the native-BCE allocation comparisons in v0.9.5 are not interchangeable. [T07–T10]

Independent populations, optimizer seeds and repeated timing calls measure different sources of variation. An interval crossing zero does not establish equivalence. The v0.9 Banknote check uses one split and five optimizer seeds; MLP4 has the four original features while most other arms use two PCA inputs, so that contrast does not match input information. [T09, pp. 4, 16]

Source lineage and implemented differences

The architecture proposal v0.3 and the architecture specification patch v0.3.1 have both been recovered. The patch is documented in its addendum. This edition retains the proposal's abstract and imports explicitly identified patch clarifications. Study v0.3 names a separate completed experiment. This reconstruction does not retrospectively make an executed study identical to an earlier proposal. [S4–S6]

Table 3 Architecture-to-implementation bridge

Topic	Proposed record	Implemented or revised record
Reference version	The preserved architecture proposal v0.3 dated 18 October 2026. [S4]	Study v0.1 names architecture specification patch v0.3.1, which is available with its revision addendum. [S5, S6; T01, p. 2]
Circuit gradients	Both proposal editions describe automatic differentiation. [S4, S5]	Study v0.1 used exact parameter-shift gradients for circuits and analytic MLP backpropagation. [T01, p. 3]
Protocol completeness	v0.3 left several schedule and starting-state details open. [S4]	v0.3.1 clarifies pairing, full-pass quantum epochs, routing keys, caps and validation-only acceptance. These were proposed defaults, not tuned results. [S6]
Adaptation state	Candidate and accepted histories are separate. [S4]	v0.3.1 retains an unresolved treatment-specific adapter preparation choice. No completed adaptation experiment is claimed here. [S5, §5C; S6]

4 Classical and quantum circuit evaluation methods

The specification below is proposed architecture and protocol, not a reconstruction of every completed study. The architecture specification patch v0.3.1 clarifications are carried forward explicitly; study-specific departures remain in each report. Classical language and adaptation components and the bounded two-qubit experiment have separate evidence boundaries. [S4–S6; T01–T10]

Decisions to freeze before each proposed experiment

The listed defaults are proposals, not fitted optima. Development-selected inputs must be recorded before their experiment runs; language and multimodal extensions require separate design. The single-task text baseline, adaptation study and independent circuit experiment have different prerequisites.

Decision area	Status before execution	Detailed rules
Data and task	Freeze exact releases, splits, label/head contract and any ordered adaptation task sequence.	Classical data and study design; Appendix A, Replay and adaptation contract.
Semantic priority	Develop rubric anchors and evidence rules; retain the stated unknown-evidence policy and measure occupancy.	Priority classes and their evidence; Feasibility before a priority experiment.
Model selection	Proposed limits: up to six configurations and paired seed sets. Freeze permitted configurations and deterministic ranking/tie rules.	Tuning and seeds; Primary decision rule.
Adaptation	Starting/preparation rule and acceptance thresholds remain to freeze; register retry/stop triggers within the stated aggregate limits.	Appendix A, Adaptation starting states and decisions.
Resources	Name hardware, accounting unit and numeric end-to-end cap from development; include required preparation and all attempts.	Exposure and resource comparisons; Cost ledger.

Classical reference model

The text pilot uses the following proposed defaults. A label is a training or evaluation field, never an inference input. A priority rating is required for the scheduling experiment, not for generating classifier logits. B is batch size, T token length and K the prediction-label count.

Table 4 Proposed classical reference interfaces

Component	Proposed default and interface
Tokenizer and backbone	Training-only BPE with 8192 vocabulary entries; T at most 256. Six pre-normalised encoder blocks, width 256, four heads, feed-forward width 1024, GELU and learned positions.
State 3	Validate and tokenise raw input. Training adds labels, class, evidence and source IDs. Inference supplies text and mask; random routing derives its own label-independent key from input text (Appendix A).
State 2	Shared output H [B,T,256]; masked mean h [B,256]. Cosine routing uses a separately normalised copy of h and 18 frozen training centroids.
State 1	One selected residual adapter per example. Bottleneck width 16 with ReLU and biases; output is raw h plus adapter(h). Linear head produces [B,K] logits.

Component	Proposed default and interface
Replay store	2000 records total. Training-only unique IDs with fixed accepted snapshots and per-label reservoir quotas; protocol in Appendix A.

Listing 1. The forward path returns logits and the route without changing accepted training state. [S5]

```
tokens, mask = validate_and_tokenize(raw_input)
H = backbone(tokens, mask)
h = masked_mean(H, mask)              # raw pooled vector
u = normalize_for_routing(h)          # separate copy
k = cosine_argmax(u, frozen_centroids)
z = h + up[k](relu(down[k](h)))
logits = classifier(z)
return logits, route=k
```

Use numerically stable cross-entropy directly from logits. Fit centroids in evaluation mode on training-only embeddings after the uniform model warm-up. Break exact ties by the smallest index. A zero-norm route uses adapter 0 and emits a flag; an empty post-validation input is rejected. Appendix A fixes numerical and optimiser defaults.

The backbone can move relative to frozen centroids. Measure route changes on a fixed training probe set every 100 updates. A separately declared frozen-backbone control freezes the entire predictive backbone after warm-up; it does not add a second routing encoder. Report total, active and trainable parameters, stored bytes and memory. The nonlinear adapter is not a LoRA rank-16 update. [R2, R11]

Priority classes and their evidence

Priority class c is distinct from prediction labels, modality, attention heads and qubits. Apply access and required integrity checks first. Eligible records receive four ratings in the interval zero through one: validity, provenance completeness, task relevance and annotation consistency. All four ratings and evidence references are required for a numeric score.

$$q = \frac{r_{valid} + r_{prov} + r_{task} + r_{ann}}{4}$$

If scoring evidence is incomplete, store q as unknown and assign provisional C8 with a reason. Do not substitute zero or average only the available ratings. Provisional records remain eligible under the proposed sampler. Report provisional C8 separately from assessed low-score C8. Synthetic origin is an independent field.

Table 5 Semantic evidence-priority classes and sampling weights

Class	Complete evidence score	Weight
C1	0.875 to 1 inclusive	8
C2	0.750 to below 0.875	7
C3	0.625 to below 0.750	6
C4	0.500 to below 0.625	5
C5	0.375 to below 0.500	4

Class	Complete evidence score	Weight
C6	0.250 to below 0.375	3
C7	0.125 to below 0.250	2
C8	Below 0.125 or provisional unknown	1

For example, ratings 0.8, 0.9, 0.7 and 0.6 give q = 0.75 and C2. The same record with one missing rating instead has unknown q and provisional C8. Record scorer, rubric version, evidence IDs and adjudication.

Feasibility before a priority experiment

AG News does not supply these ratings. During development, a designated annotation lead must publish anchors and evidence requirements, estimate annotation effort and double-score a training-only audit sample with adjudication. Report agreement, class occupancy, score distributions and provisional proportions before freezing the rubric. Do not force eight occupied bins with arbitrary quantiles. If every eligible record receives one class, this sampler reduces to uniform and cannot test semantic priority. [R8]

For held-out subgroup reporting, apply the frozen rubric through an evaluation-only process after the procedure is fixed. These annotations cannot affect training, checkpoint selection or predictions. The neutral quantum study instead uses independent synthetic class IDs; they do not implement this quality rubric.

Priority sampling and controls

Let N be the number of eligible records in the selected source pool, n_c its class-c count and W the sum of weights over nonempty classes. Each draw is independent with replacement. Select a uniform record with probability one half; otherwise select a class in proportion to weight and then a uniform record in that class.

$$P(i) = \frac{1}{2N} + \frac{w_{c(i)}}{2W n_{c(i)}}$$

$$P(class\ c) = \frac{n_c}{2N} + \frac{w_c}{2W}$$

This allocates a class-level budget. A record in a rare low-priority class can be drawn more often than one in a common high-priority class. For example, with 900 C1 and 100 C8 records, P(C1 record) is about 0.000994 and P(C8 record) about 0.001056. The C1 class still receives about 89.4 percent of draws. Log class mass and per-record repetition.

Table 6 Proposed classical sampling protocol

Control	Exact policy
Uniform records	Independent replacement draws from all N records.
Uniform processing classes	Choose a nonempty processing class uniformly, then a uniform record. This is not prediction-label stratification.

Control	Exact policy
Proposed mixture	The two-branch probability above; w_c equals 9 minus c.
Shuffled priority	Freeze a seeded permutation of training-record priorities within the source pool, preserving its class histogram.
Alternative per-record weighting	Separate optional hypothesis: mix uniform sampling with record weights proportional to w_c. It does not replace the proposed class-budget policy.

Sampler RNG streams are recorded by run, episode and source pool. A wholly empty training pool terminates the round with a reason. During adaptation, first choose new data with probability 0.8 or replay with probability 0.2 for each of 64 batch positions, then apply the treatment sampler within that pool. This yields 12.8 replay records per batch in expectation, not an exact count. Log realised counts.

Replay-off and empty-replay controls draw all positions from new data. An empty new-data pool ends the adaptation episode; it cannot silently become replay-only training. Keep the accepted replay snapshot fixed throughout candidate training. Match these source rules across replay controls.

Table 7 Synthetic circuit allocation namespace

Synthetic allocation IDs	Re-uploading layers	Meaning
C1 / C2	4	Independent synthetic metadata; no semantic quality assessment.
C3 / C4	3	Independent synthetic metadata; no semantic quality assessment.
C5 / C6	2	Independent synthetic metadata; no semantic quality assessment.
C7 / C8	1	Independent synthetic metadata; no semantic quality assessment.

Prediction labels, semantic evidence-priority classes and synthetic allocation IDs are distinct namespaces. Historical C identifiers are retained for traceability. Eight synthetic IDs select four layer policies; they do not represent eight assessed quality levels in the circuit benchmark. [S5; T01, p. 3]

Bounded quantum model

Use two data qubits and classical class metadata, with no ancilla or coherent class register. The neutral benchmark supplies two features and a synthetic class ID. Q-Sphere validates these inputs; Cube selects a re-uploading layer count; Sphere records the resulting prediction, cost and classical optimisation update. This is a separate experiment interface, not an established plug-compatible replacement for the text backbone. [R3, R4]

$$d(c) = 5 - \left\lceil \frac{c}{2} \right\rceil, \quad c \in \{1, \ldots, 8\}$$

C1/C2 use four layers, C3/C4 three, C5/C6 two and C7/C8 one. All classes have the same operation permissions. Adjacent pairs therefore implement four execution policies, not eight. The selected layers are the prefix of one shared four-layer parameter bank. Here d counts re-uploading layers; compiled circuit depth is reported separately.

Table 8 Eight-entry operation registry for the bounded circuit proposal

ID	Registered operation	Proposed use
OP01	Hadamard H	Reserved and disabled.
OP02	Pauli X	Reserved and disabled.
OP03	RY rotation	Feature, trainable and fixed readout rotations.
OP04	RZ rotation	Trainable rotation.
OP05	CX	Control q0 and target q1 in each layer.
OP06	SWAP	Reserved and disabled.
OP07	Z-basis measurement	Terminal readout of q0.
OP08	Reset to zero	Preparation between executions.

OP01–OP08 is distinct from the historical gate-family categories. OP03–OP05 are active unitary types; OP07 and OP08 are nonunitary lifecycle operations. Repeated permissions define 64 entries, not 64 algorithms. Any future eight-policy mechanism must specify and compare its changes.

Circuit sequence

Prepare both qubits in zero. Each layer applies RY(pi times x0) on q0 and RY(pi times x1) on q1, trained RY then RZ on each, then CX(q0,q1). After the last CX apply fixed RY(pi/2) on q0. Its outcome-one probability is the prediction; this readout can use both features at one layer.

The bank allocates 16 angles and activates 4d per record, with 6d + 1 source rotations and d CX gates. Fixed-layer models optimise 4d angles. Adjacent RY gates may merge. Counts imply neither independent degrees of freedom nor universality. [R3, R14]

Quantum data and matched controls

Generate 8000 feature pairs uniformly on the square from minus one to one using NumPy PCG64 seed 17. Label a pair one if the sum of squared coordinates is less than two divided by pi; otherwise zero. The disk occupies half the square in expectation. Assign independent uniform synthetic classes with seed 23. Freeze a 60/20/20 train/validation/test split stratified by label and class using seed 29.

Random classes test allocation mechanics, not semantic quality priority. Do not replace them with target-derived metadata. A later evidence-based rule must be defined from information available at inference and evaluated as a new study.

Table 9 Proposed quantum data and controls protocol

Treatment	Information and parameters
Scheduled quantum model	Two features and selected d; one shared 16-angle prefix bank. Report exposure per layer.
Fixed quantum models	Four separately trained models with d equal to 1, 2, 3 or 4 and 4d optimised angles each.
Shuffled allocation	Own shared 16-angle prefix bank. Freeze a within-split permutation of layer assignments preserving each histogram. Record seeds and assignments; this is a benchmark control, not a deployment rule.
Matched-information MLP	Two features plus one-hot selected layer count; 6 inputs, 16 ReLU units, sigmoid output and biases: 129 parameters.
Richer-metadata MLP	Two features plus one-hot class ID; 10 inputs, 16 ReLU units and sigmoid output: 193 parameters. It receives more metadata than the circuit.
Optional small MLP	6 inputs, 2 ReLU units and sigmoid output: 17 parameters. Similar counts do not imply equal expressivity or compute.

The matched MLP receives the circuit's exact d assignment, including a separate matched control for shuffled allocation. Neither MLP is a parameter match. Scheduled and shuffled quantum treatments each own one shared 16-angle prefix bank; weights are never shared across trained treatments. Match initialisation, optimiser, batches, selection and tuning budget, pairing initial draws and record-order streams where meaningful. Fixed-layer treatments independently optimise 4d angles.

Training and checkpoint rules

Use one seeded shuffled full training pass without replacement per epoch, batch 64, retaining a partial final batch. The proposed Adam learning rate is 0.01 with a 200-epoch cap. Select lowest validation binary cross-entropy, ties by earliest epoch. Appendix B fixes optimiser, initialisation and seed conventions; all settings are proposed, not empirical results.

For the numerical loss clip predicted probabilities to [0.0000001, 0.9999999]; report clipping frequency. Classify one at probability at least 0.5, including ties. The quantum study has its own epoch and quality contract; the classical 0.85 time-to-target rule does not apply.

Classical data and study design

Use AG News as the proposed single-task text benchmark after recording the exact release, licence/access terms, file hashes and label map. Preserve official test membership and labels. Define an exact duplicate unit by Unicode NFKC normalisation, case folding, whitespace collapse and hashing title plus body. Quarantine training candidates matching test content. Do not use test labels or outcomes for this contamination check. [R8]

Group remaining exact training duplicates before selecting a group-stratified validation partition targeting 10 percent; record its actual size and any conflicting-label exclusions. Freeze thresholds before outcomes are examined. Fit tokenizer, rubric and centroids only on the resulting training partition. Any near-duplicate analysis or cleaned test set must be a separately named protocol. The Cubit Benchmark requires recovered records and ground truth before use.

Table 10 Proposed classical study design protocol

Study	Treatments and purpose
Minimum text comparison	Uniform/shared adapter, priority/shared adapter, uniform/routed adapters, priority/routed adapters. These four cells test two factors before replay is relevant.
Routing controls	Dense backbone, shared adapter, content routes and stable per-record random routes using training-only utilisation probabilities.
Priority controls	Uniform records, uniform nonempty processing classes, proposed mixture and fixed shuffled-priority metadata.
Adaptation factorial	Priority on/off, routed/shared adapters and replay on/off: eight cells after the task sequence is specified. Match acceptance policy across cells.
Later sensitivities	1, 18, 36 and 72 adapters; output-layer and retrieval studies. Disclose parameter and budget changes.

For stable random routing, estimate content-route frequencies on training embeddings at the frozen warm-up checkpoint. Freeze these probabilities and map a versioned hash of canonical input text through their cumulative distribution using the fixed route seed (Appendix A). Labels, priority ratings and split identity never enter the key. Repeated normalised inputs retain the same route, including new inference inputs; report realised utilisation.

Equal-backbone treatments share tokenizer, backbone shape and warm-up checkpoint. An approximately equal-total-parameter dense control targets a 5 percent tolerance using development-selected dimensions, with its own initialisation and warm-up. If no valid shape meets tolerance, report the nearest feasible mismatch. Neither condition is automatically equal compute.

Development selects hyperparameters and eligible comparators. Final settings and five paired seeds are frozen before testing. Keep record access, labels, precision and tuning opportunities comparable; count scoring and routing overhead.

5 Correctness, resource measurement and reproducibility

Quantum checks and resource reporting

Construct an independent 4 by 4 matrix reference from explicit gates, without reusing the candidate gate helper. Compare all eight classes over nine equally spaced values of each feature and five Uniform[-pi, pi] parameter banks (Appendix B): 3240 bounded comparisons. Include encoding, trainable gates and readout, then repeat after compilation.

Use complex128 arithmetic and maximum elementwise error at most 0.0000000001 after one global-phase alignment. Account for compiler wire permutations before comparison. Check prediction probabilities to the same tolerance. Appendix B defines basis, multiplication order and alignment. This is finite numerical coverage, not proof over all continuous inputs and parameters.

Independent invariants

For one layer define A0 = pi times x0 plus its trained RY angle and A1 analogously; b0 and b1 are the trained RZ angles. Under Appendix B conventions the analytical readout is:

$$p_1 = \frac{1 + \sin(A_0)\cos(b_0)\sin(A_1)\cos(b_1)}{2}$$

Use this expression as a specification check. Identical features and parameters must give identical predictions for C1/C2, C3/C4, C5/C6 and C7/C8 under the unshuffled mapping. This invariant does not apply when per-record shuffled assignments select different layer counts. Different layer counts are different models, not equivalent circuit rewrites.

Measurement and costs

Statevector training uses classical automatic differentiation and zero physical measurement shots. Evaluate frozen checkpoints with 256, 1024 and 4096 simulated Bernoulli shots per test record, with five shot seeds nested inside each trained checkpoint. These are repeated readouts, not 25 independent training runs. Finite-shot inference does not establish finite-shot trainability.

Report qubits, allocated/active/optimised angles, source and compiled gate counts, compiled depth, circuit evaluations, submitted circuit batches and total shots separately. Record compiler version, basis RY/RZ/CX, optimisation level, mapping and seed. Include preparation, readout, encoding, routing, compilation, optimisation and classical costs in named categories.

A balanced class distribution gives 2.5 layers per example; independent uniform classes give that expectation. Report the realised mean from actual records and training exposure. This count is not a runtime gain. Reset and measurement need separate channel or distribution checks outside the unitary matrix comparison. [R5]

The Gu preprint and author repository illustrate independent task-specific evaluation. Their restricted evaluator is not a verifier for this classifier. Keep exact resource counts, sampled-input checks and broader mathematical proof distinct. [R6]

Exposure and resource comparisons

Table 11 Proposed measurement and metrics protocol

Comparison	Held fixed	Report separately
Fixed exposure	Processed non-padding tokens or records, declared in advance	Updates, unique records, repetition, coverage and padding cost. Specify which exposure unit is primary.
Fixed updates	Optimiser update count and global batch policy	Tokens, elapsed time and device-hours; step costs can differ.
Fixed resources	Predeclared elapsed-time or aggregate-device-hour cap	Budget boundary, evaluation/checkpoint costs, stop rule and unfinished updates.
Time to target	Quality target and confirmation rule	Full time to a qualifying checkpoint and every missed target.

Choose elapsed time or aggregate device-hours and a numeric cap from development measurements on named hardware; freeze both before final runs. Apply it to required preparation, warm-up, training, validation and checkpoint work charged to the run. Finish an in-progress update safely and log overrun. Work completed after the cap cannot turn a censored run into target success. Equal updates do not imply equal resources.

For primary classical fixed-budget quality, use the latest scheduled checkpoint whose updates, required validation and save all finish within every registered cap. Use the same rule for all treatments. Report no eligible checkpoint explicitly. A post-cap evaluation or shutdown save is a separately costed diagnostic. Test reporting is charged separately and cannot change checkpoint eligibility.

Cost ledger

Separate common dataset preparation; method-specific rating, tokenisation and centroid work; per-run training, validation and checkpoint saves; tuning; and total project expenditure. Record human annotation effort. End-to-end per-method comparisons charge their required preparation and shared warm-up even if cached; report amortised and actual project totals separately to avoid double counting reused work.

Define device-hours as the sum of participating-device allocated elapsed intervals inside the named run boundary, including reserved idle time. Report measured utilisation separately. Energy requires a stated meter and system boundary; financial cost requires rates. Neither follows from device-hours alone.

Tuning and seeds

Use development seeds 101–103 and up to six configurations per treatment. Select one configuration using only development validation, then evaluate final seeds 201–205. Do not pick a best seed. Batch permutations may differ by sampler but shared initial states remain paired where meaningful.

A fair efficiency comparison freezes an eligible control before final runs. For development eligibility, require all three seeds to reach the registered validation target and median target-checkpoint macro-F1 at least that target. Select the lowest median end-to-end target time; break ties by higher mean target-checkpoint validation F1, then fixed control ID. If none qualify, revise the development task/target or report fixed-budget results. Eligible control IDs and exclusion reasons are recorded.

Metrics and failure aware decisions

Macro-F1 averages over a fixed prediction-label list without frequency weighting; undefined per-label scores are zero. Report per-label scores and support, plus processing-class subgroups separately. Language modelling uses normalised NLL/perplexity and rare-token coverage; retrieval declares queries, gallery, relevance and direction. [R9]

Time and quality from the same checkpoint

The proposed classical target is validation macro-F1 at least 0.85 for three consecutive evaluations, every 100 global updates including model warm-up. End time when the third qualifying evaluation and its checkpoint save finish. Pair that time with test quality from exactly that checkpoint. Later best-validation or final-budget quality belongs in separate comparisons. MLPerf motivates time-to-quality measurement, not these Q-Pathformer thresholds. [R10]

The classical cap is 11000 total updates. Target success requires the third qualifying validation and checkpoint save to finish within both update and registered end-to-end resource caps (the resource comparisons subsection). Otherwise report censoring, numerical failure or infrastructure interruption as applicable. Keep incident-policy retries and all original costs visible.

Primary decision rule

A four-of-five outcome is a preliminary feasibility signal for each method. Apply the proposed primary 15 percent median time-reduction criterion only when all five final runs of both Q-Pathformer and the frozen comparator reach the target. Otherwise report success fractions, censored outcomes and quality-resource curves with no headline speedup ratio.

$$R_T = 100 \times \left(1 - \frac{median(T_{QPF})}{median(T_{control})}\right)$$

$$S_T = \frac{median(T_{control})}{median(T_{QPF})}$$

R_T is percentage time reduction and S_T the speedup factor; 15 percent reduction is about 1.176 times speedup. A promising quality-preserving efficiency result also requires mean test macro-F1 at the target checkpoints to be no more than 0.01 below the control. This descriptive criterion is not statistical noninferiority. Five successes do not establish population reliability.

Report every seed and the mean and standard deviation of quality. Bootstrap paired test records or source groups conditional on each checkpoint, separately from variation across training seeds. Do not retune or select releases using test performance. If the common warm-up reaches the target, it cannot demonstrate a routing or scheduling benefit; revise during development or use fixed-budget outcomes.

For adaptation, apply the validation-only selection and retention rules in Appendix A. Report selected-checkpoint quality but charge the full selection horizon and all attempts, even if that checkpoint is earlier. Classical fixed-budget quality uses the resource comparisons subsection's latest eligible checkpoint; quantum selection uses its separate validation-BCE rule.

Integrity checks identify the supplied files. They do not establish predictive benefit or full reproduction from curated downloads. Individual Code releases identify selected implementations and prerequisites; trained checkpoints, datasets, environments and full operational archives remain separately retained where indicated. This editorial consolidation ran no new training, test rescoring, simulator experiment or physical-QPU job. [T01–T10]

6 Limitations and next experiments

The main unresolved question is how the integrated architecture behaves beyond the tested components. Semantic-priority work first needs a usable rubric, real annotation evidence and meaningful class occupancy. Adaptation needs a defensible task sequence, accepted starting states, replay and selection rules. Proposed numerical defaults are not empirical optima. [S5, S6]

Routing studies should compare a learned gate with fixed routing, parameter-matched output correction and strong global controls. Broader populations and tasks are needed before transferring conclusions to larger models. The current component evidence does not establish general quantum advantage or a need for physical quantum execution. [R15; T03–T09]

A physical-device pilot should start with a frozen predictor and a narrow fidelity or measurement question. Qualify the current backend, compiled circuit, shot budget and saved classical reference. Hardware inference, hardware training and an advantage claim require different evidence, including classical costs. The study series reports classical and simulated execution, not completed physical-QPU results. [R4, R5, R7; T01–T10]

AI-assisted circuit discovery is a possible separate research direction. Gu and colleagues use LLM-proposed circuits with an independent task-specific evaluator for reversible-algorithm tasks. That preprint and evaluator do not verify this classifier programme or establish a completed Q-Pathformer circuit-search result. [R6]

7 Conclusion

Q-Pathformer provides an architectural framework for organising learning decisions and a study programme for testing selected mechanisms within that framework. The completed work supports useful effects under specific routing and readout conditions, identifies faster classical evaluation routes for selected fitted circuits, and retains clear cases in which the proposed method does not establish an advantage over a stronger control. These outcomes guide the next research decisions. The priority is to test defined mechanisms on broader evidence, assess the integrated adaptation design, and qualify any physical quantum experiment against a frozen classical reference and an explicit resource budget. [T01–T10]

8 References

External references provide methodological context; the T-series reports support this project's empirical claims. Private repository links require authorised access while review continues. S2 and S3 are privately retained antecedents; S4–S6 are preserved owner-review inputs. They are not additional included study papers.

[R1] Grave, E., Joulin, A., Cissé, M., Grangier, D. and Jégou, H. (2017). Efficient softmax approximation for GPUs. Proceedings of ICML, PMLR 70, 1302–1310. https://proceedings.mlr.press/v70/grave17a.html

[R2] Fedus, W., Zoph, B. and Shazeer, N. (2022). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR 23(120), 1–39. https://jmlr.org/papers/v23/21-0998.html

[R3] Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E. and Latorre, J. I. (2020). Data re-uploading for a universal quantum classifier. Quantum 4, 226. DOI 10.22331/q-2020-02-06-226. https://arxiv.org/abs/1907.02085

[R4] Cross, A. W. et al. (2022). OpenQASM 3: A broader and deeper quantum assembly language. ACM Transactions on Quantum Computing 3(3). DOI 10.1145/3505636. https://arxiv.org/abs/2104.14722

[R5] Burgholzer, L. and Wille, R. (2022). Handling non-unitaries in quantum circuit equivalence checking. Proceedings of DAC, 529–534. DOI 10.1145/3489517.3530482. Author preprint arXiv:2106.01099 (2021). https://portal.fis.tum.de/en/publications/handling-non-unitaries-in-quantum-circuit-equivalence-checking-2/ Author preprint: arXiv:2106.01099

[R6] Gu, A., Kemeny, A. H., Yelin, S. F. and Lukin, M. D. (2026). Discovering state-of-the-art quantum circuits with large language models. arXiv:2610.22367, submitted 15 October 2026. Preprint. Evaluator and circuit repository: github.com/andigu/uncompute. https://arxiv.org/abs/2610.22367 Evaluator: author repository

[R7] Beverland, M. E. et al. (2022). Assessing requirements to scale to practical quantum advantage. arXiv:2211.07629. Preprint. https://arxiv.org/abs/2211.07629

[R8] Zhang, X., Zhao, J. and LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. Advances in Neural Information Processing Systems 28. Author version arXiv:1509.01626. https://arxiv.org/abs/1509.01626

[R9] Scikit-learn developers. f1_score documentation. Macro averaging, explicit labels and undefined-score conventions. Accessed 30 October 2026. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

[R10] Mattson, P. et al. (2020). MLPerf Training Benchmark. Proceedings of Machine Learning and Systems 2. https://proceedings.mlsys.org/paper_files/paper/2020/hash/411e39b117e885341f25efb8912945f7-Abstract.html

[R11] Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M. and Gelly, S. (2019). Parameter-Efficient Transfer Learning for NLP. ICML, PMLR 97, 2790–2799. https://proceedings.mlr.press/v97/houlsby19a.html

[R12] Chaudhry, A., Rohrbach, M., Elhoseiny, M., Ajanthan, T., Dokania, P. K., Torr, P. H. S. and Ranzato, M. (2019). On Tiny Episodic Memories in Continual Learning. arXiv:1902.10486. Preprint. https://arxiv.org/abs/1902.10486

[R13] Bengio, Y., Louradour, J., Collobert, R. and Weston, J. (2009). Curriculum Learning. Proceedings of ICML. DOI 10.1145/1553374.1553380. https://ronan.collobert.com/pub/matos/2009_curriculum_icml.pdf

[R14] Schuld, M., Sweke, R. and Meyer, J. J. (2021). The effect of data encoding on the expressive power of variational quantum machine learning models. Physical Review A 103, 032430. DOI 10.1103/PhysRevA.103.032430. https://arxiv.org/abs/2008.08605

[R15] Bowles, J., Ahmed, S. and Schuld, M. (2024). Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059, version 2. Preprint. https://arxiv.org/abs/2403.07059

[S1] Siljeg, A. (2024). Q-Pathformer: Multi-state Machine Learning & LLM Training. QPATH-ARCH-001. First publication recorded as 5 December 2024; 26-page architecture paper. Included in the repository documentation, with original notices. https://github.com/qpathformer/qpathformer-classical-research-v1.0/blob/7c05ea9712d4b21a3b16fd1258f2731972e344c2/docs/architecture/QPATH-ARCH-001.pdf

[S2] Siljeg, A. Q-Pathformer: Steps to F1 Benchmarks & Foundation Code. Historical article dated 20 July 2025; assessment additions dated 18 October 2026, retained in Q-Pathformer_Steps_to_F1_Assessed_Draft.docx. Privately retained project source, not a paper in this collection.

[S3] Siljeg, A. (2026). Q Pathformer Research Consolidation: Usable foundations and highly experimental directions. Version 0.1, 18 October 2026. Q-Pathformer_Consolidated_Research_Assessment_v0.1.docx. Privately retained source.

[S4] Siljeg, A. (2026). Q-Pathformer Architecture Update: Research proposal and experimental specification. QPATH-ARCH-002, version 0.3, 18 October 2026. Preserved source manuscript; retained in the owner review inputs.

[S5] Siljeg, A. (2026). Q-Pathformer Architecture Update: Research proposal and experimental specification. QPATH-ARCH-002, version 0.3.1, 18 October 2026. Recovered specification patch; retained in the owner review inputs.

[S6] QPATH-ARCH-002 Final Review and Revision Addendum (2026). Proposal version 0.3.1, 18 October 2026. Privately retained decision and provenance record accompanying S5; not empirical evidence.

[S7] Siljeg, A. (2026). Q-Pathformer Routed States: Classical Models and Quantum Circuit Simulation. Consolidated Research Findings, v1.0. Draft main paper in this collection. Main paper PDF

[T01] Siljeg, A. (2026). Q-Pathformer: Local benchmark and IBM readiness. QPATH-RPT-002, study-v0.1, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.1

[T02] Siljeg, A. (2026). Q-Pathformer: Improvement study. QPATH-RPT-003, study-v0.2, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.2

[T03] Siljeg, A. (2026). Q-Pathformer: Shallow circuits and routing study. QPATH-RPT-004, study-v0.3, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.3

[T04] Siljeg, A. (2026). Q-Pathformer: Routing and circuit contribution study. QPATH-RPT-005, study-v0.4, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.4

[T05] Siljeg, A. (2026). Q-Pathformer: Flexible readouts and classical controls. QPATH-RPT-006, study-v0.5, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.5

[T06] Siljeg, A. (2026). Q-Pathformer: Harder tasks and faster inference. QPATH-RPT-007, study-v0.6, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.6

[T07] Siljeg, A. (2026). Q-Pathformer: Routing capacity and classical deployment. QPATH-RPT-008, study-v0.7, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.7

[T08] Siljeg, A. (2026). Q-Pathformer: Priority training and routed models. QPATH-RPT-009, study-v0.8, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.8

[T09] Siljeg, A. (2026). Q-Pathformer: Plan B findings: Geometric routing, learned routing and capacity controls. QPATH-RPT-010, study-v0.9, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.9

[T10] Siljeg, A. (2026). Q-Pathformer: ETEL training allocation study. QPATH-RPT-011, study-v0.9.5, report edition 1. Supporting report included unchanged in the Papers collection; original study dates and limitations remain in the report. https://github.com/qpathformer/qpathformer-classical-research-v1.0/releases/tag/study-v0.9.5

Appendix A Classical training, replay and implementation defaults

These proposed details retain the recovered v0.3.1 clarifications. They are not a statement that every component study executed this complete protocol. The adapter starting-state choice and adaptation sequence remain research decisions. [S5, S6]

Training phases and accepted state

Table 12 Training phases and update budgets

Phase	Trainable modules	Proposed schedule
Uniform model warm-up	Backbone and classifier; no adapters	1000 updates; 100-update linear LR warm-up, then cosine decay. All phases use batch 64, peak LR 0.0003 and endpoint zero.

Phase	Trainable modules	Proposed schedule
Main text phase	Backbone, classifier and batch-active adapters	Reset optimiser moments; 10000 updates with 1000-update LR warm-up, then cosine decay. Fit centroids before this phase.
Adaptation episode	Classifier and batch-active adapters; backbone and centroids frozen	2000 updates total across at most three attempts. Reset each attempt optimiser/LR; nominal 2000-step horizon, 200-step warm-up. See the replay and adaptation rules below.

These are proposed schedules, not tuned results. Equal-backbone treatments share the same warm-up checkpoint within each seed and charge its cost. A differently sized dense model has its own seeded warm-up. Charge centroid fitting only where required. Evaluate every 100 updates, including model warm-up; adaptation uses attempt-local evaluation counters and a separate aggregate episode budget. Appendix A fixes LR endpoints and horizons.

At the batch level, active adapters are the union of selected routes. Parameters and optimiser moments for every inactive adapter must remain unchanged, including no AdamW decay. Use separate parameter groups or an explicitly masked update; an inactive slice in a packed tensor must not drift. Hyperparameters, precision and initialisation are fixed in Appendix A.

Accepted and candidate snapshots

An accepted snapshot records backbone, adapter and head weights; tokenizer and centroid versions; class/rubric policy; replay version; optimiser and scheduler state; configuration and RNG state. Clone it for a candidate episode. Ordinary gradients update candidate state, not the accepted snapshot.

On acceptance, commit the chosen candidate checkpoint and staged replay/policy versions atomically. On rejection, preserve metrics and predictions in the append-only ledger and restore the accepted snapshot. Rejected records cannot enter replay or alter priorities. Use a predeclared distinct next-attempt seed derived from run ID, episode and attempt. Appendix A limits attempts and aggregate work; rollback never refunds consumed budget. All attempt costs count.

Feedback runs only between declared adaptation episodes. Sphere supplies candidate errors and record IDs; Cube aggregates route/error evidence; Q-Sphere stages independent annotation or quality evidence for the next episode. Confidence alone cannot promote a class. No feedback changes frozen test data or retroactively edits accepted run records.

Replay and adaptation contract

The replay capacity is 2000 records total. Divide capacity by the declared label count using integer quotas; allocate remainders in sorted label-ID order. Unused quotas remain unused. Insert each unique eligible training ID once in stable ID order using a recorded per-label reservoir RNG. Repeated sampling visits do not count as new admissions.

The initial accepted model admits its eligible training records. During a candidate episode, retain the committed buffer and stage unique eligible new records for insertion only after acceptance. Reject duplicate IDs, record parent IDs for transformed samples, and exclude all validation and test records. The first pilot uses a fixed label universe. A changing universe requires a separately specified quota and head-migration protocol before that study starts. [R12]

Table 13 Adaptation experiment contract

Protocol field	Decision required before adaptation runs
Task sequence	Select a defensible dataset and ordered tasks. Prefer common-label domain adaptation where source metadata supports it; no domain sequence is inferred from AG News.
Prediction space	Freeze common label list and one shared head. Any class-incremental variant must state union/head expansion and access to task identity.
Populations	Freeze training, validation and untouched test IDs for every task, plus source/duplicate grouping.
Initial state	Reuse starting checkpoints only across compatible model shapes. Declare common backbone/head preparation and the adapter rule; later episodes use each treatment's accepted history (Appendix A).
Budget and selection	Aggregate 2000-update episode cap across at most three attempts, plus one episode resource cap. Every 100 attempt updates, select highest new-task validation F1 satisfying retention; earliest checkpoint breaks ties.
Acceptance	Use frozen validation populations: meet the new-task target; old-task F1 drop at most 0.01 for each prior task and their unweighted mean, relative to the accepted pre-episode state.
Matched controls	Same buffer, sampling mixture, attempt limits and acceptance rule, unless that element is the declared treatment.

The sequence and metadata remain pending. Within the classical track, single-task baseline implementation can proceed; the semantic-priority comparison also needs rubric evidence. The independent circuit pilot has its own readiness steps. Report individual old-task drops and their unweighted mean; an average cannot conceal a failure on one task.

The replay on/off factor belongs to this adaptation study, where replay examples exist. The separate acceptance-policy experiment compares controlled acceptance with unconditional candidate selection; it must not be silently included as a fourth factor in the three-factor design.

Adaptation starting states and decisions

At the common initial setup, reuse the same accepted checkpoint only within identical model shapes. For shared-versus-routed adapters, register the common backbone and head state and a treatment-specific adapter initialisation or transfer rule. Save checkpoint hashes, paired seeds and preparation costs. No reviewed project decision fixes that adapter rule; it remains pending before adaptation runs. After the common setup, each treatment continues from its own accepted history.

Checkpoint selection, every new-task threshold and every retention threshold use frozen validation populations. Immediately before an episode, score the accepted model on those same prior-task validation populations to establish retention baselines. Test populations are reserved for reporting after the decision and cannot determine candidate acceptance, retry choice or checkpoint selection.

An episode permits at most three candidate attempts and 2000 optimiser updates in aggregate. Each attempt may consume only the remaining episode update and resource allowance; there is no separate full allocation for each retry. A first attempt that exhausts either allowance leaves no

budget for another attempt. Rollback restores accepted model state, but the episode cost and consumed-update counters remain outside that snapshot.

Reset optimiser moments and the attempt-local LR counter for each attempt. Its nominal schedule remains 2000 updates with 200 warm-up updates, even when the remaining episode allowance ends it earlier. Do not rescale the horizon after early termination. Register any early-stop or retry trigger before runs; rejection alone creates no additional budget. Candidate checkpoints must finish their required validation and save within both episode caps to be eligible.

Count all attempts, validation, saves, selection and rollback overhead. Selecting an earlier checkpoint does not shorten the charged selection horizon. If no eligible checkpoint meets all validation thresholds, preserve the accepted state and report episode failure. The numerical resource cap and adaptation starting-state rule must be frozen in the experiment manifest before this study starts.

Reproducibility configuration

The following defaults close implementation ambiguities. Dataset-dependent fields listed in the adaptation protocol still need evidence before the corresponding experiments start.

Table 14 Proposed classical implementation defaults protocol

Area	Proposed configuration
Text processing	BPE vocabulary 8192 including PAD, UNK, BOS and EOS. Fit training text only. Truncate content to 254 tokens and add BOS/EOS; right-pad. Empty content is rejected. Pool non-PAD, non-special content tokens.
Encoder	Pre-LayerNorm, epsilon 0.00001; GELU feed-forward; dropout 0.1 on attention and residual branches. Bidirectional padding mask for classification. Learned positions 0–255. Linear layers and LayerNorm have biases.
Initialisation and precision	Linear/embedding weights normal with SD 0.02; biases zero, LayerNorm scale one. Adapter down weights same rule; up weights and both adapter biases zero. Float32 reference; no mixed precision in initial comparison.
Optimiser	AdamW beta values 0.9/0.999, epsilon 0.00000001, weight decay 0.01 excluding biases and LayerNorm; gradient norm cap 1.0. Phase schedules and attempt-local counters are specified in the schedule and routing conventions subsection.
Centroids	Unit-normalised training means, spherical k-means++ seed derived from run seed, maximum 100 iterations or centroid displacement below 0.000001. Normalise centroids after updates. Refill empties with farthest training example, ties by ID.
Routing numerics	Use h divided by max(norm(h), 0.00000001) only for routing. If norm is below that threshold, route to adapter 0 with a flag. Cosine ties choose lowest index.
Drift probe	Freeze up to 1000 eligible training IDs by seeded sampling. Every 100 updates report fraction changing route from warm-up assignment and from previous probe, plus cluster utilisation.
Required manifests	Code commit, package versions, hardware, precision, data and split hashes, label universe, all RNG streams, configurations, optimiser state, accepted/candidate lineage and checkpoint hashes.

Softmax is for reported probabilities; cross-entropy consumes logits. Do not feed normalised routing vectors to the residual adapter. Group decisions for a batch before updating; inactive adapters retain both parameters and optimiser moments. Inference runs in evaluation mode with dropout disabled.

Schedule and routing conventions

New proposed schedule defaults in architecture specification patch v0.3.1 are peak learning rate 0.0003, cosine endpoint zero and batch 64 for every classical phase. Warm-up, main and adaptation-attempt horizons H are 1000, 10000 and 2000 updates, with linear warm-up lengths W of 100, 1000 and 200 respectively. These values complete the specification; they were not selected by experiments.

For one-based local update t at or below W, use LR = 0.0003 × t/W. Thereafter use LR = 0.0003 × [1 + cos(pi × (t − W)/(H − W))]/2 through t = H. Reset the local counter and optimiser at each phase or candidate attempt. Planned horizons remain fixed under early stopping. Adaptation attempts share the aggregate 2000-update allowance; consumed episode counters never reset on rollback.

Random routing uses key version qpf-text-route-v1. Start from the full input text supplied to tokenisation, before truncation; for AG News this is title, one newline, then body. Apply Unicode NFKC, case folding and whitespace collapse, trim, then encode as UTF-8. This canonical text key is separate from provenance record IDs and does not contain labels, priorities or split membership.

Compute SHA-256 over the ASCII decimal nonnegative route seed, one zero byte and canonical UTF-8 bytes. Interpret the 32-byte digest as a big-endian unsigned integer J. Select the first route k for which J/2^256 is below its cumulative training-derived probability, using adapter-index order; the final cumulative boundary is one. Record the normalisation/runtime version, seed, probabilities and mapping version. Identical canonical inputs retain a route, and inference needs no external record ID.

The run manifest stores phase and attempt counters, full scheduler settings, all RNG stream names and seeds, resource units and scope, and checkpoint identities. Adaptation also needs its task sequence, common backbone/head preparation and treatment-specific adapter rule. Those evidence-dependent choices remain pending rather than being supplied by these numerical defaults.

Appendix B Quantum conventions and random streams

Quantum numerical conventions

Use basis order |q0 q1>, with q0 the first tensor factor and amplitude indices 0, 1, 2, 3 corresponding to 00, 01, 10, 11. Vectors are columns; later gates multiply on the left. CX has q0 as control and q1 as target. An RY rotation has rows [cos(theta/2), minus sin(theta/2)] and [sin(theta/2), cos(theta/2)]; RZ is diagonal with exp(-i theta/2) and exp(i theta/2).

The probability of q0 equal to one is the sum of squared amplitude magnitudes at indices 2 and 3 after the fixed readout rotation. Record the whole two-qubit state and selected marginal in numerical tests. Use float64 real parameters and complex128 matrices. These conventions control the analytical expression and compiler permutation adjustment.

Global phase and wire layout

Let U be the independent reference and V the candidate matrix after mapping compiler wire order back to the declared logical order. Compute z as the trace of U conjugate-transpose times V. If its magnitude exceeds 0.000000000001, multiply V by the complex conjugate of z divided by its

magnitude and compare to U. Otherwise flag alignment failure and inspect the mismatch. For equivalent four-dimensional unitaries the trace magnitude is four.

The reference constructs gates independently from explicit matrices. It includes feature preparation rotations, trainable layers and the fixed readout rotation, but excludes reset and terminal measurement. Validate reset as a preparation channel and terminal readout through its output probabilities. Future dynamic circuits require appropriate nonunitary semantics. [R4, R5]

Seeds and budgets

For each numerical-check seed 301–305, use a separate NumPy PCG64 generator to draw 16 independent angles uniformly from minus pi to pi. Store each full bank in layer-major, then qubit q0/q1, then RY/RZ order. Save the vectors and hashes. These wider check settings are separate from training initialisation. The feature grid is minus one, minus 0.75, minus 0.5, minus 0.25, zero, 0.25, 0.5, 0.75 and one.

Record compilation target and final wire map, optimisation level and seed before final comparisons. Source rotations can merge, so do not infer compiled depth from layer count. A shot is one sampled measurement outcome; a submitted circuit batch can contain many circuits and shots; an exact statevector evaluation is a deterministic simulator call with no physical shots.

Finite-shot predictions use the count of outcome-one draws divided by the declared shot count. Threshold ties at 0.5 predict one. Probability clipping affects only loss numerics, with frequency reported. No training, simulator run or hardware execution is claimed by the document’s analytic or arithmetic specification checks.

Training and random streams

The following are new proposed reproducibility defaults. A quantum epoch visits every training record once in a seeded shuffled permutation without replacement. Use batch 64 and keep the final partial batch. Use the same rule for the circuit models and their MLP controls. Derive distinct, recorded streams for parameter initialisation and epoch permutations from each run seed. Allocation uses the fixed split seeds below.

All benchmark treatments use Adam with learning rate 0.01, beta values 0.9 and 0.999, epsilon 0.00000001 and zero weight decay. Keep the learning rate constant for the nominal 200-epoch cap. Initialise circuit angles independently Uniform[-0.1, 0.1]. For every MLP linear layer, sample weights independently Uniform[-sqrt(6/(fan_in + fan_out)), +sqrt(6/(fan_in + fan_out))] and set biases to zero. Record dtype and the random generator implementation in the frozen manifest.

Scheduled and shuffled circuit models use paired initial 16-angle draws and paired record-order streams for each seed, but train separate banks and checkpoints. Match all other hyperparameters and tuning opportunities. Fixed-layer models own 4d angles. When shapes differ, pair data order where meaningful and record the distinct initialisation dimensions.

Development uses seeds 101–103 and at most six configurations per method. Freeze settings before final seeds 201–205, retaining every seed. Validate after each epoch and select lowest validation binary cross-entropy, ties by earliest epoch. Charge the full training and selection horizon. Classical fixed-resource checkpoint eligibility is a separate comparison rule.

Freeze within-split shuffled allocation with seeds 401, 402 and 403 for training, validation and test. Store assignments and realised histograms. The matched-information MLP receives those exact assignments. Use shot seeds 501–505 nested within each selected training checkpoint; repeated shot

readouts do not become independent training runs. Record generator versions and saved random inputs wherever exact replay is required.

Appendix C Companion tasks and lineage

Language model output layer

Classification cannot establish large-vocabulary savings. Use a separate causal decoder with training-only tokenizer and partition fitting. Routing at token t may use only the causal hidden state through t. Compare full softmax, adaptive softmax and a proposed 18-shard hierarchical head with disjoint shards covering every token. [R1]

The head predicts a normalised shard distribution and a normalised token distribution within each shard. Their product defines a full-vocabulary distribution. Preserve the local token index in the likelihood computation.

```
log_s = log_softmax(shard_head(causal_hidden))
s, j = token_to_shard_and_local_index[target_token]
log_w = log_softmax(token_head[s](causal_hidden))
loss = -log_s[s] - log_w[j]
```

Targets select the evaluated likelihood term during training; they are not inference inputs. Ancestral sampling selects a shard and then a token. Exact best-token decoding may examine multiple shards and must count that work. Any top-shard approximation reports excluded mass and quality separately. Compare priority-informed partitions with frequency-based ones without claiming hierarchical factorisation itself is new.

First multimodal extension

After the text pilot, use paired text-image retrieval as the first multimodal extension. Hold encoders, pairs, query/gallery membership, relevance and direction fixed. Compare uniform, source-group-balanced and priority schedules. Fully paired records already supply both modalities; modality balancing is distinct only for a later mixed or missing-modality dataset. Preserve masks and source IDs.

Retain the original text-audio, text-video, text-speech synthesis, text-action and text-sensor interfaces as later tasks. They need appropriate sequence encoders and task-specific heads, labels and metrics; they are not interchangeable agents. Projection to shared width alone does not define a complete multimodal model. [S1, pp. 6–9]

UGCCMT record contract

Table 15 Proposed companion tasks and lineage protocol

Field	Required content
Unit and Group	Record hash and parents; immutable dataset, split, label and modality identity.
Colour and Code	Display category only; transform version, implementation commit and configuration hash.
Measure	Score evidence; route observation keyed by run, checkpoint and router version; metrics and timing units.

Field	Required content
Train	Run, seed, accepted/candidate checkpoint IDs, replay version, attempt and update decision.

Route assignments may drift; they are observations, not immutable dataset identity. Record rejected inputs and updates. Report required-field completeness and logging-enabled versus logging-disabled runtime and storage. Lineage supports traceability without asserting compliance.

Appendix D Source history and planning context

The historical architecture supplies the named states, cluster notation, modality interfaces and UGCCMT identifiers. Older scalar-code demonstrations and resource projections are context, not verified neural or hardware results. This edition does not import those old performance claims. [S1–S3]

Table 16 Proposed historical architecture interpretation protocol

Original element	Decision for this proposal
Q-Sphere Cube and Sphere	Retain named responsibilities and connected forward flow. Treat feedback as evidence processing between adaptation rounds.
18 and 72 clusters	Start with 18 adapters; test other counts separately. The geometry and counts are not established optima.
144 surface spikes	Preserve as historical interface notation, not a tensor dimension or physical qubit allocation.
72 24 8 route groups	Do not infer executable rules from the inconsistent 72/24/8 and 72/48/8 source wording.
Eight classes and gate categories	Define the priority policy and a distinct OP01–OP08 registry. No one-to-one mapping to historical gate families is asserted.
UGCCMT	Use typed lineage fields and versioned observations. Governance authority remains external.

The following staging and run-count tables are preserved as planning context from the proposal. They do not describe a current commitment, a completed integrated programme or a hardware-duration estimate. Later component studies have their own budgets and results. [S5; T01–T10]

Staged research programme

The ten-week sequence is illustrative. Hardware, data annotation and the eventual paper focus determine the schedule. The single-task text result is independently useful; the circuit study can begin when its own specification and evaluator are ready. Language-model and multimodal extensions are not prerequisites for either.

Table 17 Illustrative research stages

Stage	Readiness and exit evidence
Weeks 1 and 2	Freeze data identities, rubric feasibility, label map, baseline and resource measurement boundary. Exit with runnable baseline and documented class occupancy.
Weeks 3 and 4	Implement connected states and minimum four-cell text comparison. Exit with independent metric checks, route telemetry and reproducible checkpoints.
Weeks 5 and 6	Run final text comparisons; begin adaptation only after its task contract is frozen. Exit with all outcomes, failures and uncertainty.
Parallel circuit track	Build the independent reference and numerical checks, then train the bounded pilot. Exit with correctness coverage and separate predictive/resource results.
Weeks 7 to 10 if justified	Adaptation factorial, output-layer or first multimodal extension selected by paper scope and available budget. Exit need not be a positive result.

Table 18 Illustrative development and confirmation run counts

Programme	Development upper bound	Final runs
Minimum text 4 treatments	4 × 6 configs × 3 seeds = 72	4 × 5 = 20
Adaptation factorial 8 cells	8 × 6 × 3 = 144	8 × 5 = 40
Quantum programme M methods	M × 6 × 3 = 18M	5M
Additional controls/extensions	Enumerate separately before scheduling	Five per selected treatment

These are planning counts, not resource estimates. Do not multiply six tuning configurations into every final seed. For the quantum programme, count scheduled, four fixed-layer, shuffled, matched-information, shuffled-matched and richer-metadata controls explicitly; the optional small MLP adds a method. Record actual unique runs and reused checkpoints in the ledger.

Before scheduling the full programme, estimate per-run preparation, training and evaluation time from development, multiply by the selected run counts, and record annotation effort, hardware availability and contingency. No hardware-duration estimate is invented here. More clusters or qubits are not assumed to improve the measured trade-off.

The current edition preserves the source abstract, separates the semantic and synthetic namespaces, adds the completed-study synthesis, retains the seven native mathematical expressions, and connects v0.3 to the recovered v0.3.1 patch. The old proposals remain unchanged. Editorial assistance and integrity checks do not replace independent scientific evaluation.